

Introduction to Algorithm

기초 알고리즘

Greedy 알고리즘 구현



어떤 문제가 그리디로 해결되는지 판단하기 위해선,
최적 부분 구조와 탐욕적 선택 조건을 만족하는지 확인해야 함



그리디의 조건

- **최적 부분 구조**(optimal substructure)
 - 전체 문제의 최적의 해결 방법은 각 부분 문제의 최적 해결 방법의 조합으로 이루어 진다
- **탐욕적 선택 속성**(greedy choice property)
 - 현재 상황에서의 최적 선택이 항상 전체 문제의 최적해로 이어진다



그리디 문제를 풀 때,

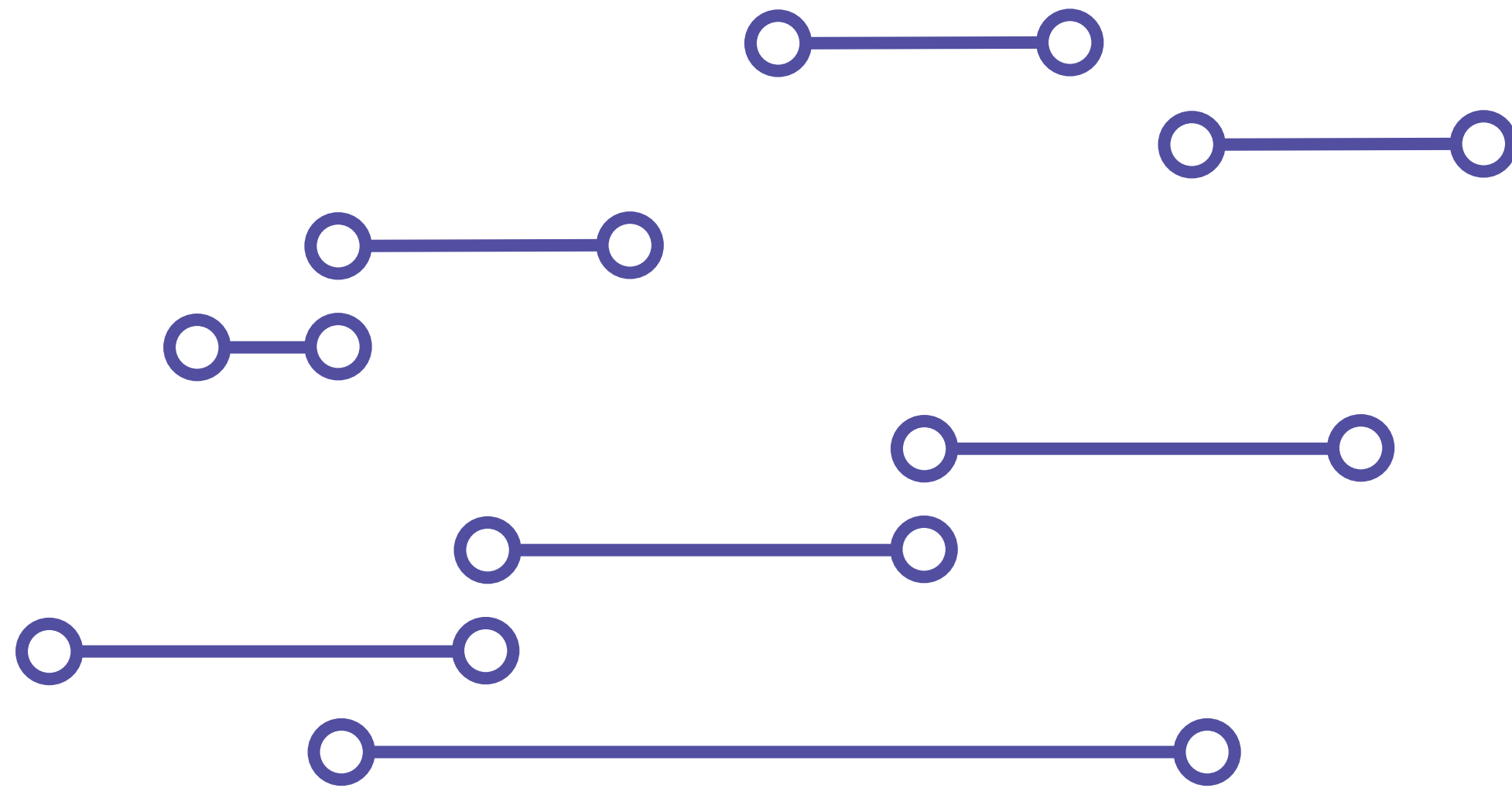
최적 부분 구조와 탐욕적 선택 조건을 만족하기 위해서

어떤 기준으로 최적의 선택을 해야 할까?

다음 문제를 풀어보며 생각해보자!



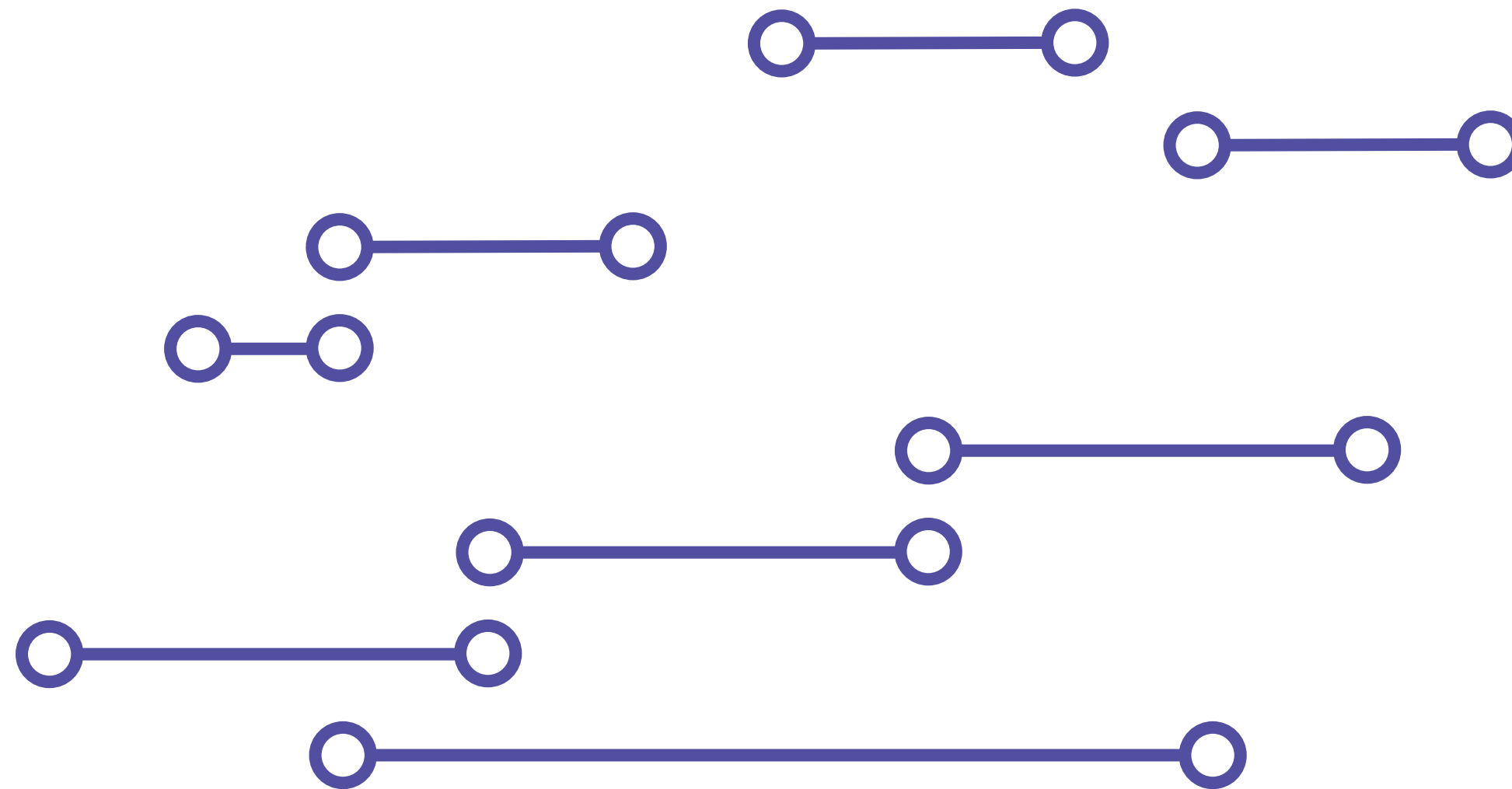
그리디 알고리즘 구현



다음과 같은 구간들이 있음



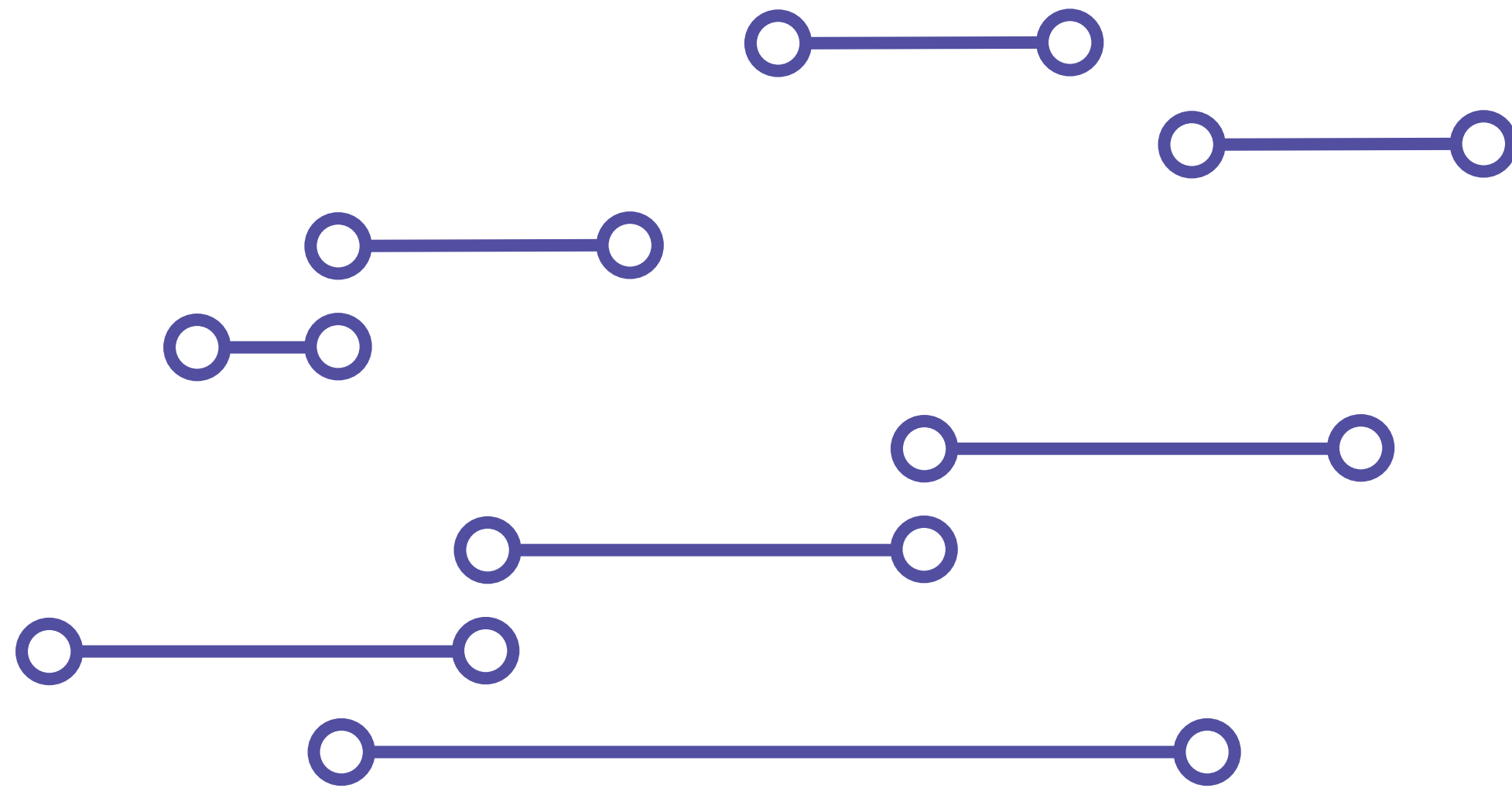
그리디 알고리즘 구현



각각의 구간은 시작점과 끝점이 있으며,
우리는 최대한 많은 구간을 선택하고자 함



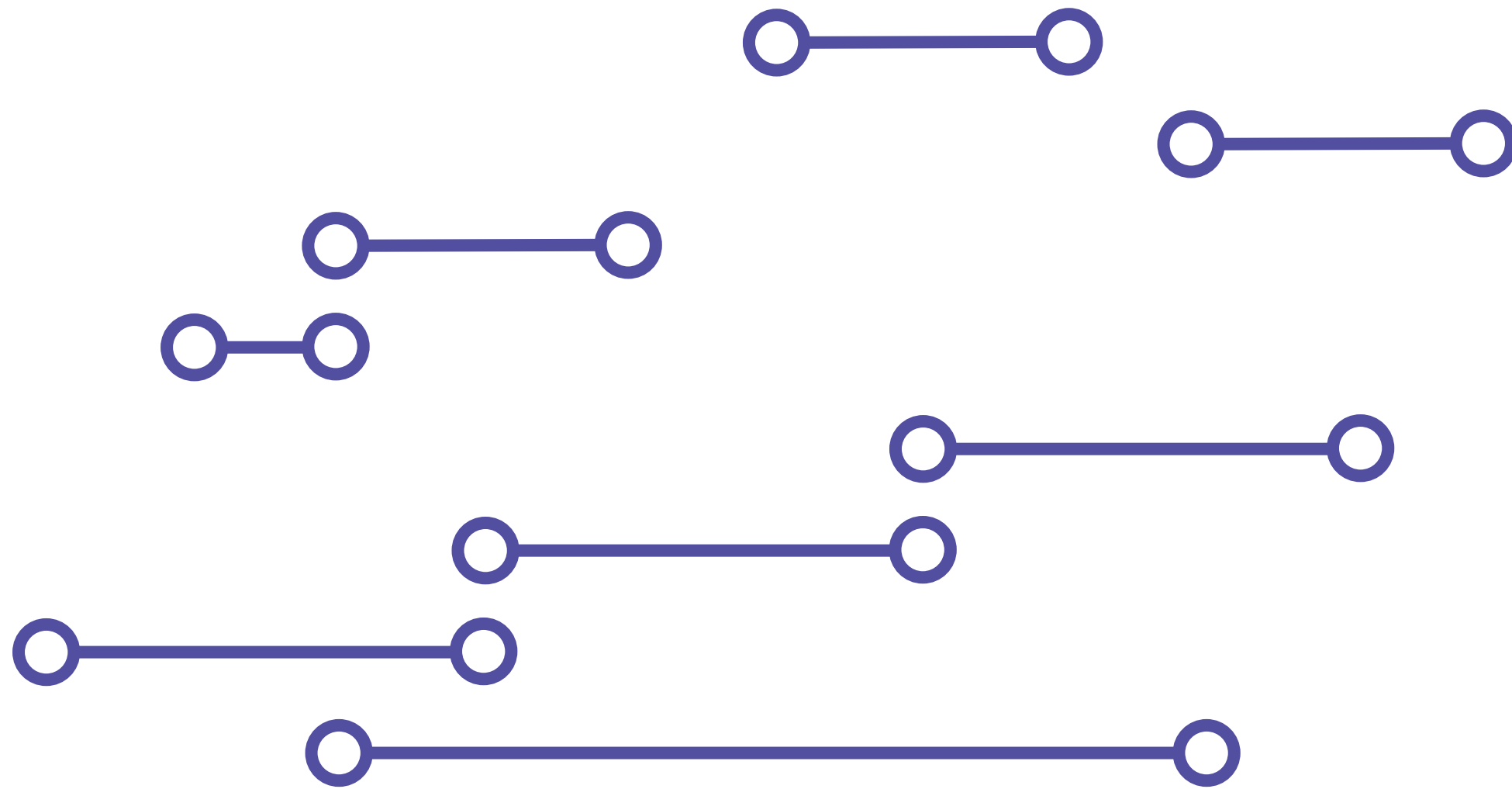
그리디 알고리즘 구현



각 구간을 선택할 때는 겹쳐선 안되나,
어떤 끝 지점과 시작 지점이 만나는 건 가능함



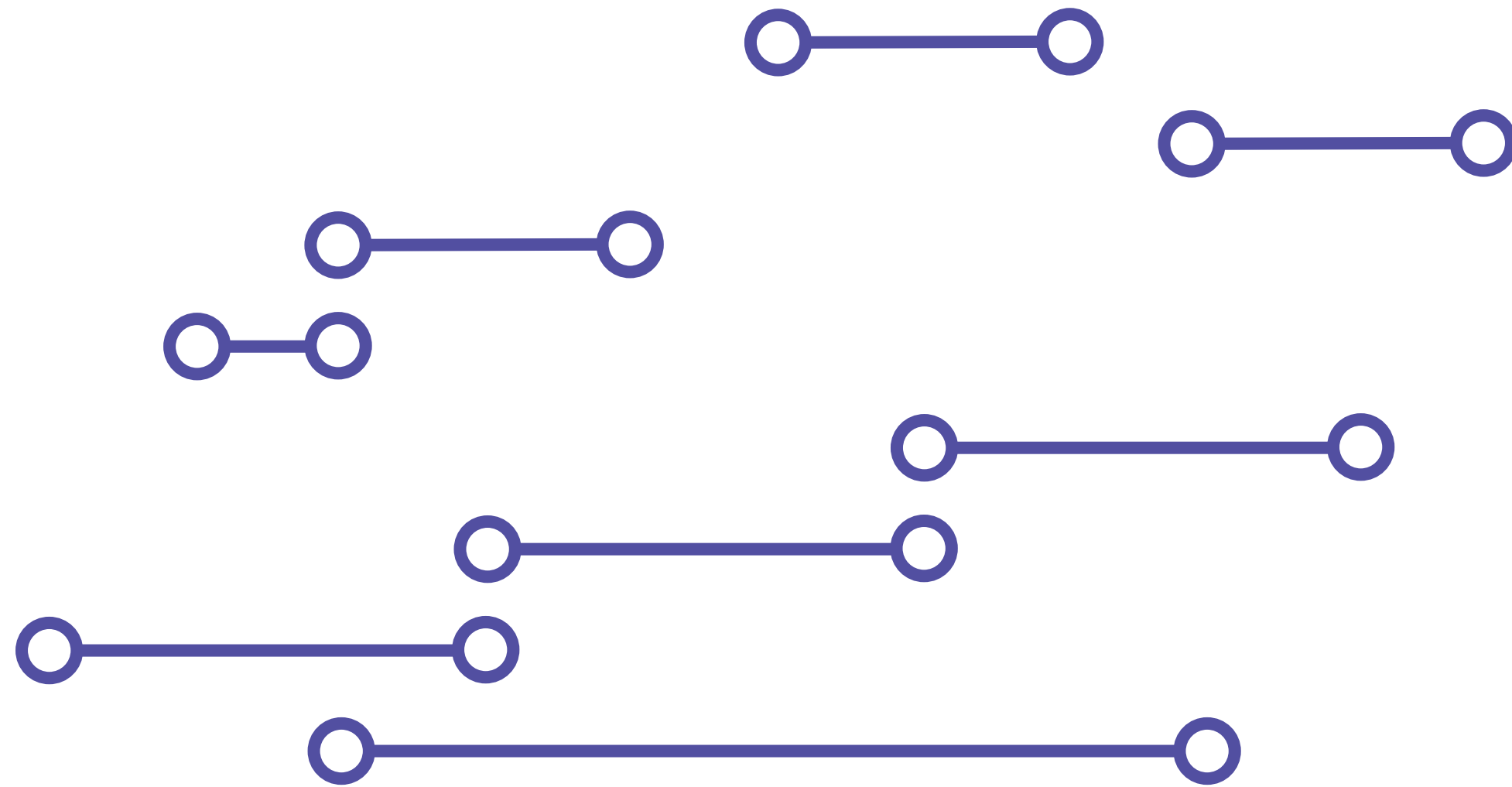
그리디 알고리즘 구현



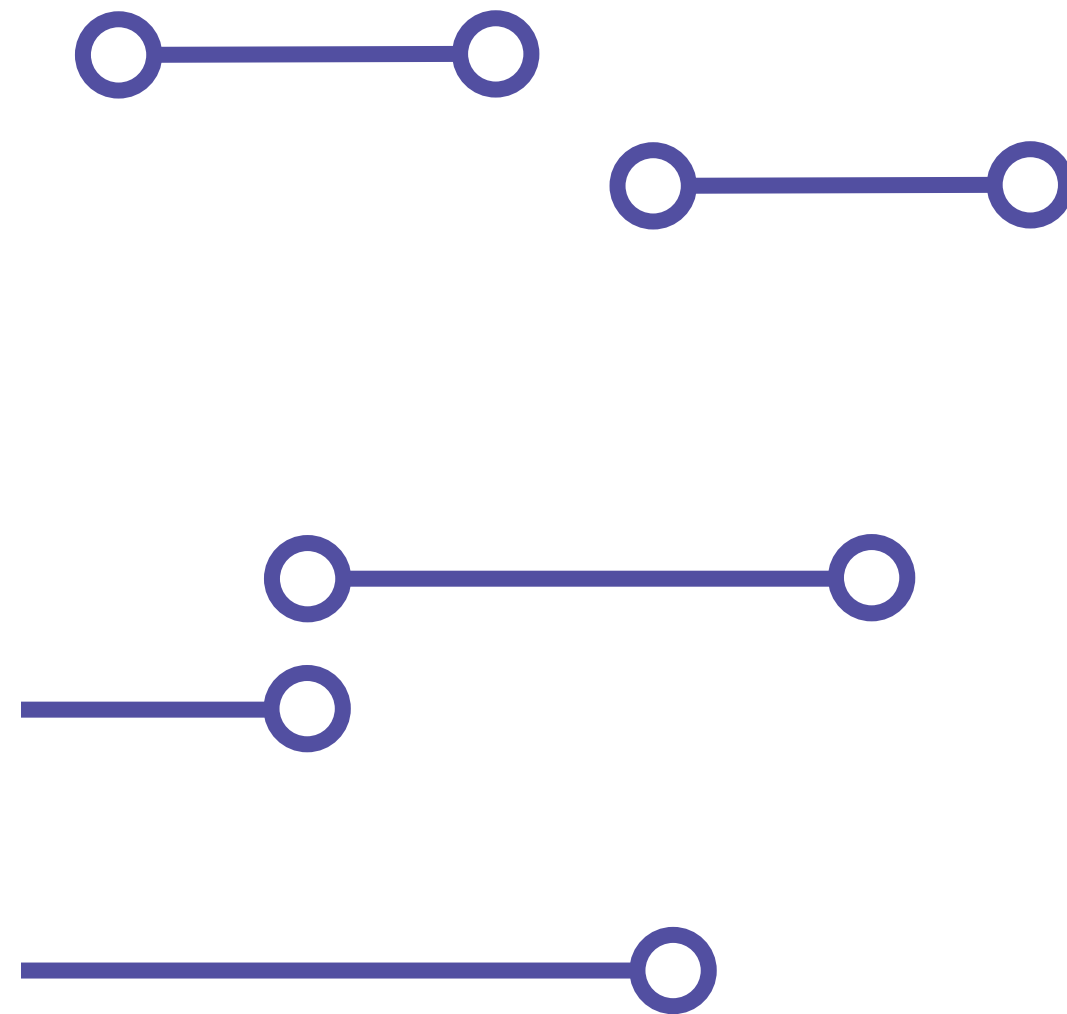
그렇다면, 어떤 방식으로 이 문제를 풀어야 할까?



그리디 알고리즘 구현

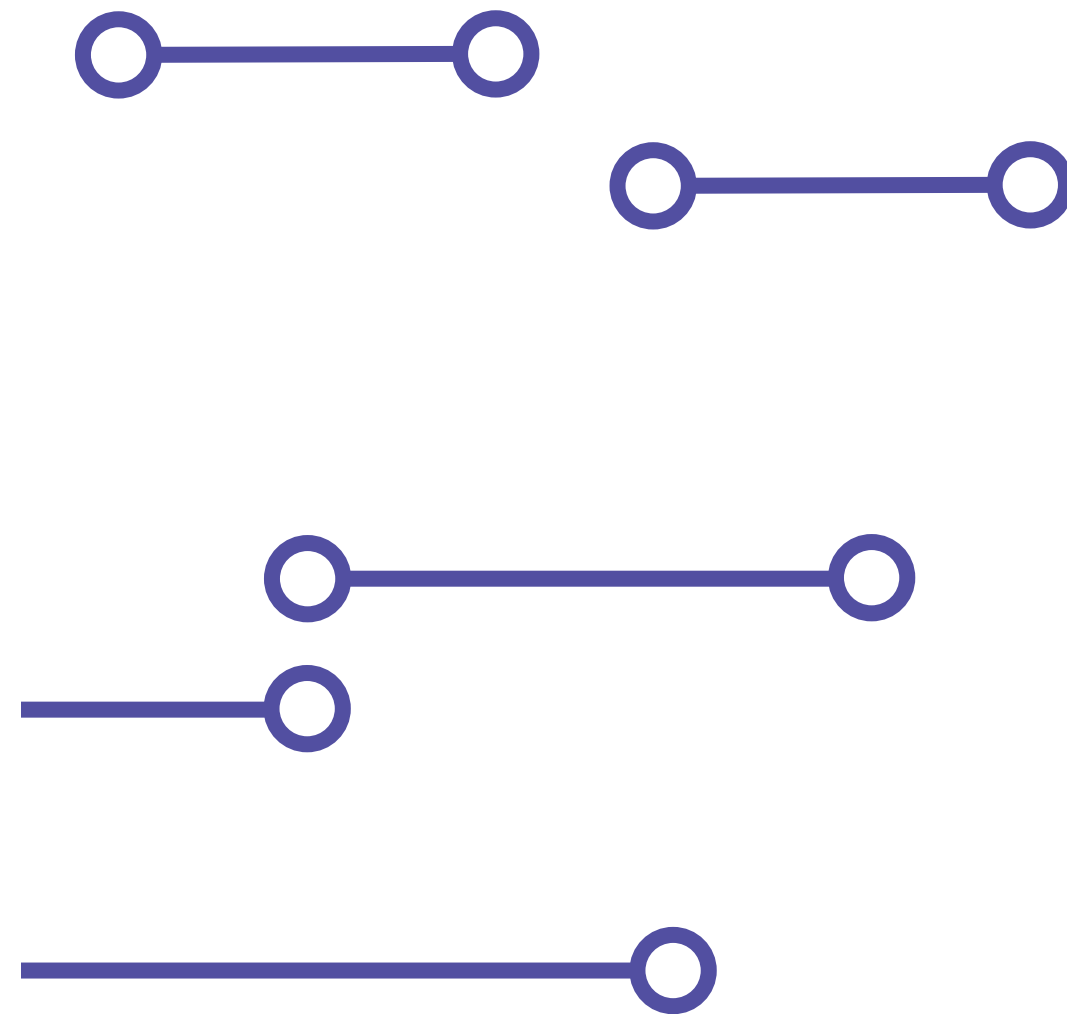


우리의 목적은 최대한 많은 구간을 고르는 것



한 가지 가정을 해보도록 하자.

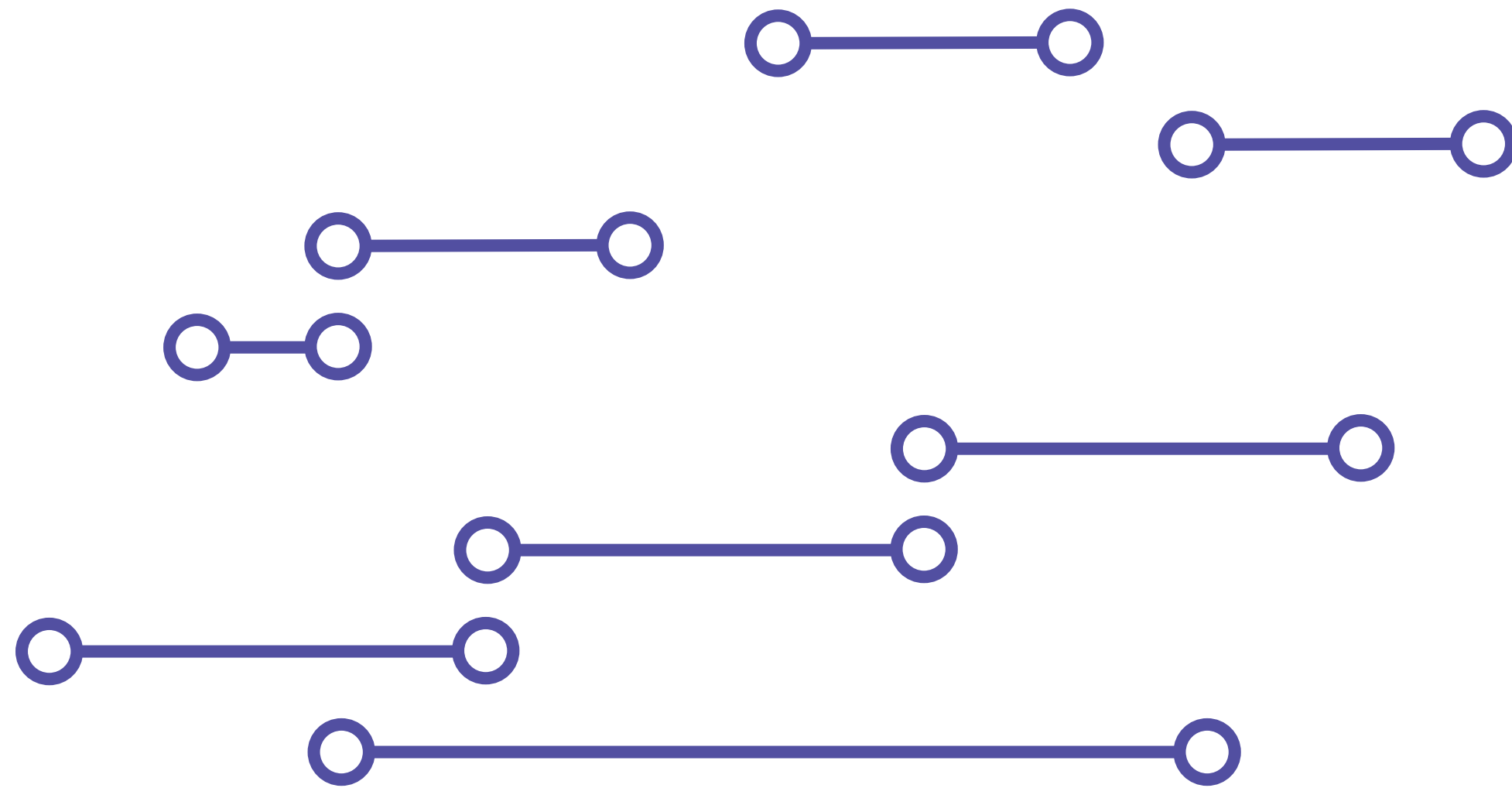
만약 앞 절반을 선택할 수 없다면 어떨까?



한 가지 확실한 건, 절반으로 줄어든다면
선택할 수 있는 구간의 수는 줄어들게 됨



그리디 알고리즘 구현

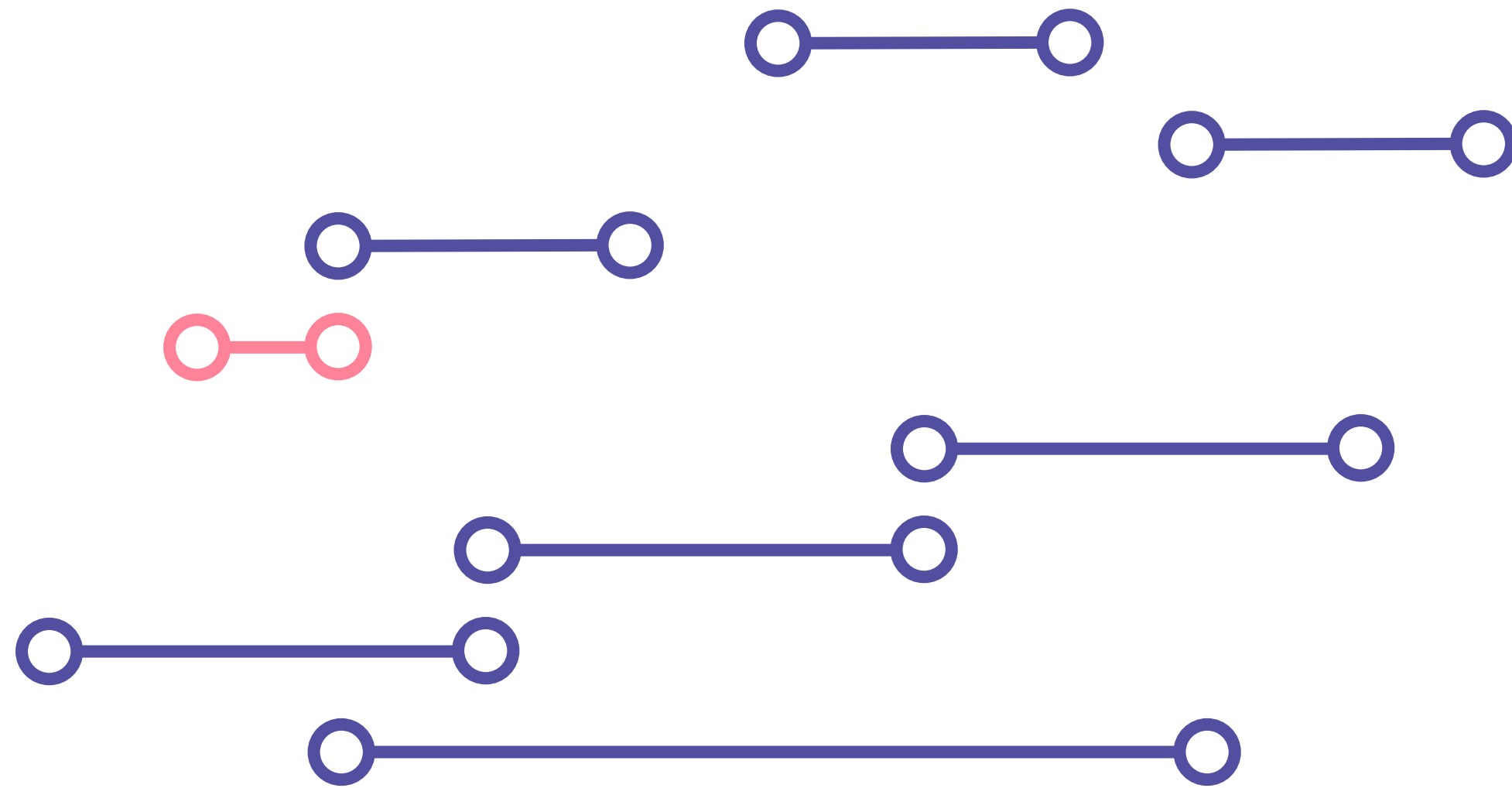


즉, 어떻게든 선택 가능한 구역을 최대화 하면

더 많은 구간을 고를 수 있음



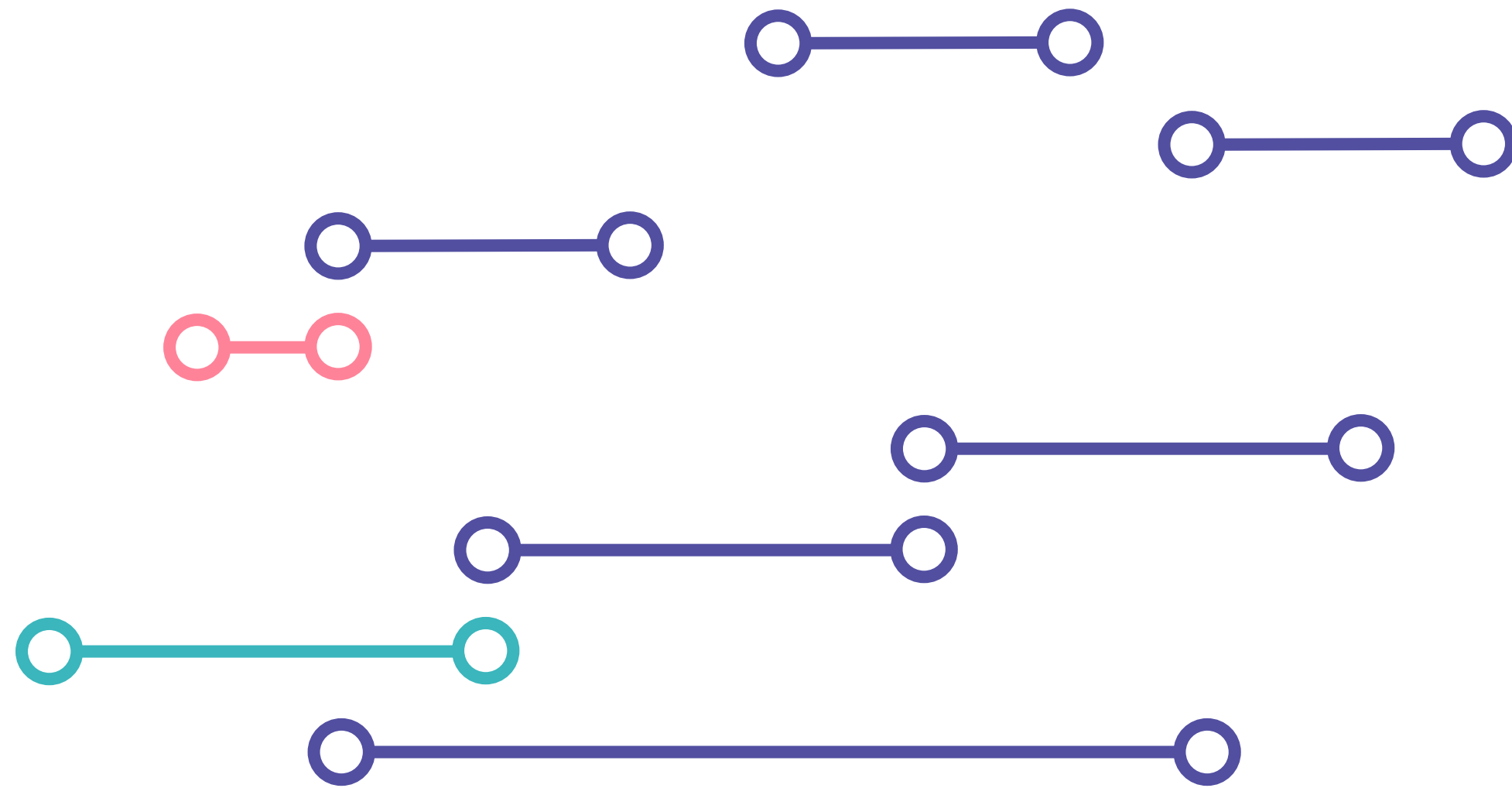
그리디 알고리즘 구현



따라서, 가장 먼저 끝나는 구간을 선택



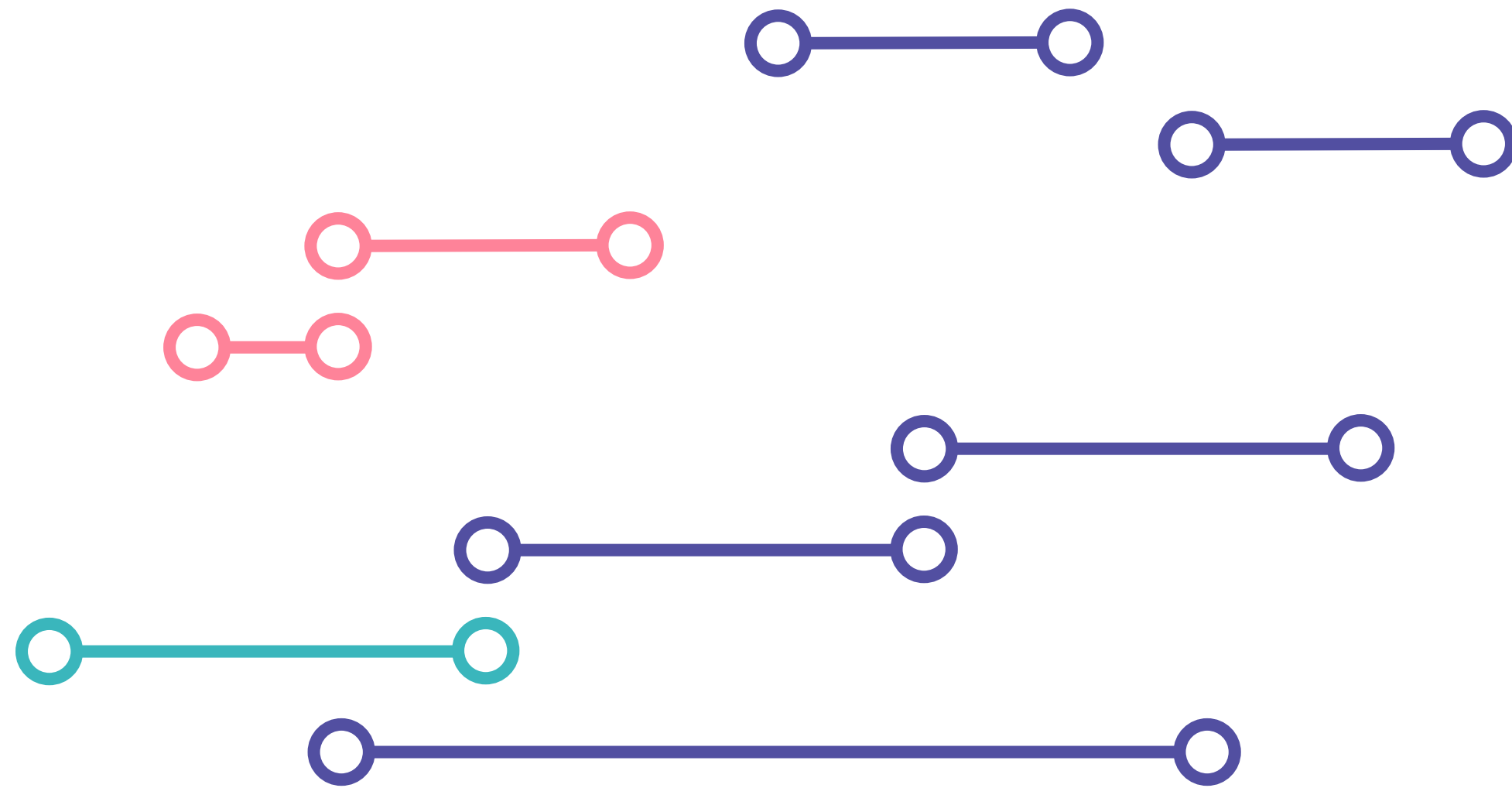
그리디 알고리즘 구현



한 구간을 선택하게 되면,
자연스럽게 겹쳐지는 구간은 선택 불가능하게 됨



그리디 알고리즘 구현



이어서, 똑같이 가장 먼저 끝나는 구간을 선택



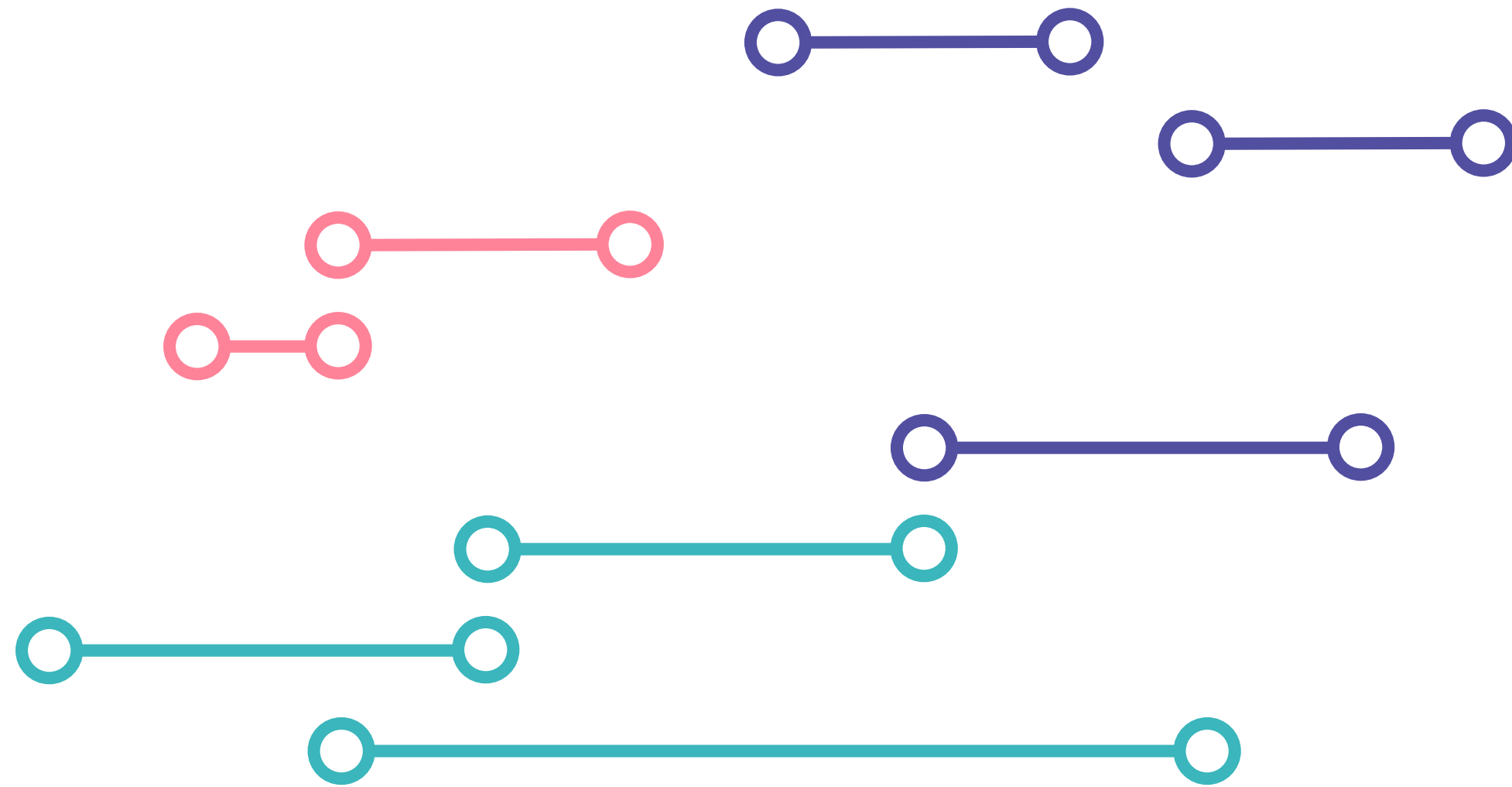
그리디 알고리즘 구현



이제 다음으로 먼저 끝나는 구간을 선택



그리디 알고리즘 구현



여기서 중요한 점은

제외된 구간을 선택하지 않도록 조심해야 한다는 것



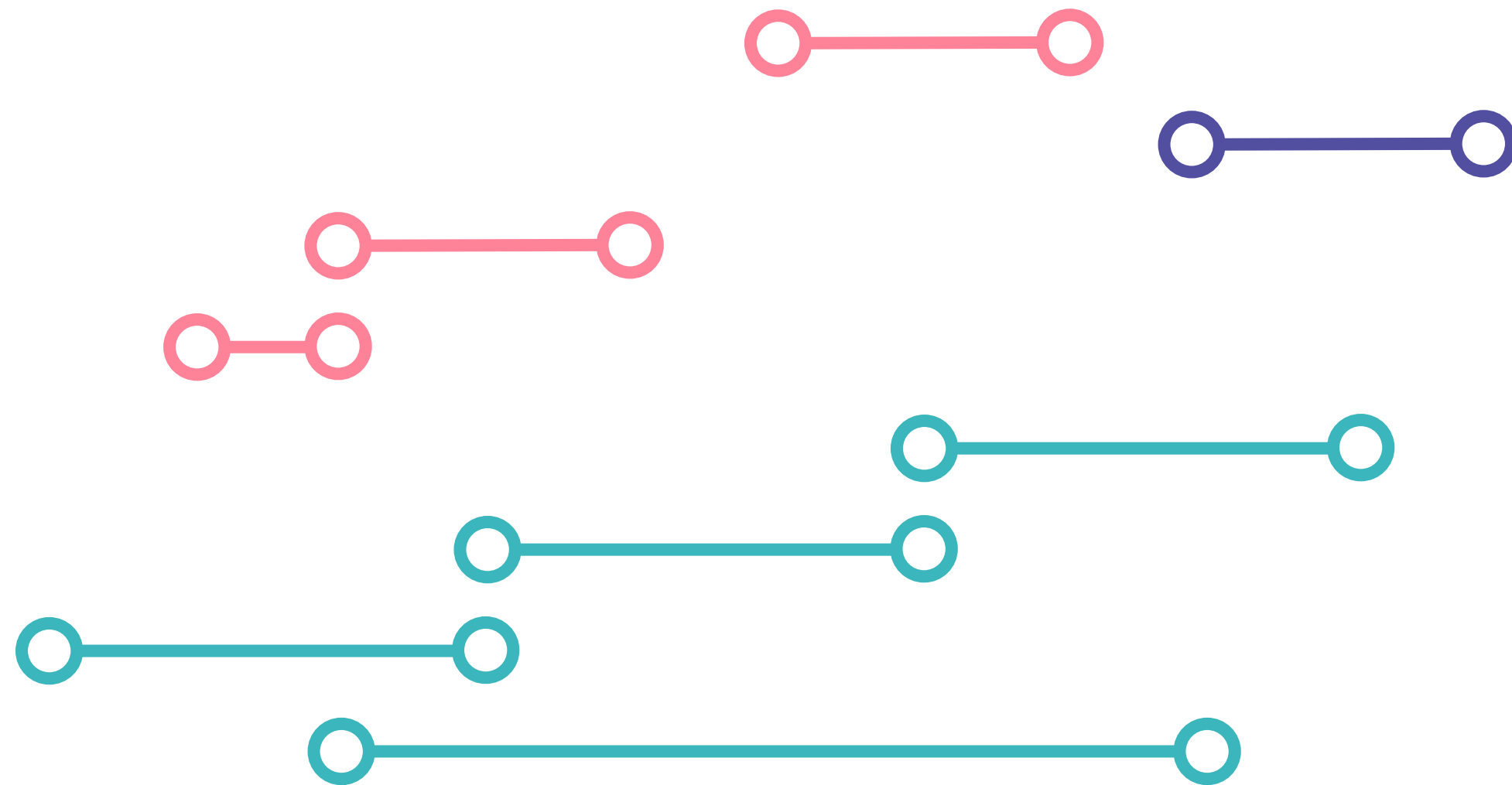
그리디 알고리즘 구현



계속해서 선택 가능한 구간이 없을 때 까지 반복



그리디 알고리즘 구현



계속해서 선택 가능한 구간이 없을 때 까지 반복



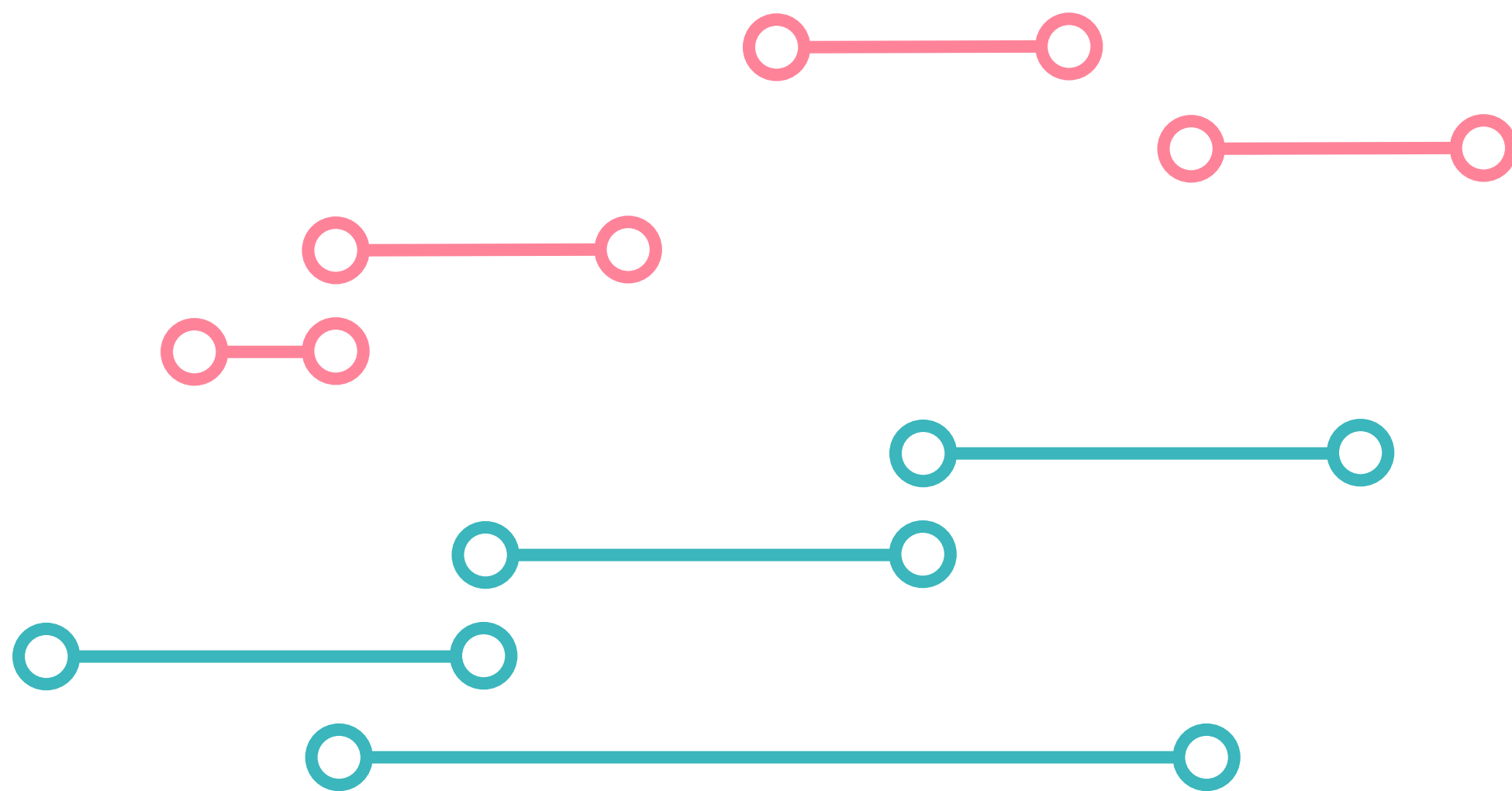
그리디 알고리즘 구현



계속해서 선택 가능한 구간이 없을 때 까지 반복



그리디 알고리즘 구현



이렇게 되면,

총 4개의 구간을 선택할 수 있음



이 문제의 경우, 종료 지점을 기준으로 정렬하게 되면
쉽게 해결할 수 있음

그리디 문제를 풀게 된다면,
반드시 어떤 기준으로 최적의 선택을 해야 할 지 잘 생각해야 함